

TASK-EFFICIENT AGENT EVALUATION USING CACHED TRACES

Anonymous authors

Paper under double-blind review

ABSTRACT

Efficient benchmarking estimates a system’s full-benchmark performance from a small number of task executions. Standard approaches rely solely on the binary correctness of executions from a collection of already-evaluated reference systems. An agentic system, however, produces a rich trace for each execution that can reveal substantially more about its capabilities. Exploiting this information requires useful trace representations. We propose representing a trace by extracting a structured, rubric-guided summary of it with an LLM and embedding the summary with a standard text encoder. These representations preserve correctness- and task-related signal while minimizing the presence of system identity-, model family-, and harness-related signals that dominate raw trace embeddings. Given this representation for each task execution from all reference systems, we construct low-dimensional representations of the systems, predict a new system’s score from its position among the reference systems, and combine the resulting geometric prediction with a two-parameter logistic item-response model. On SWE-bench Verified and Terminal-Bench 2.0, the proposed estimator reduces mean absolute error from 0.13 to 0.08 ($p < 10^{-4}$) and from 0.14 to 0.12 ($p < 10^{-3}$), respectively, when provided a single random executed task relative to an item-response model alone. These results indicate that trace geometry and correctness outcomes carry complementary predictive signal and enable more accurate benchmark score estimation from limited agent executions.

1 INTRODUCTION

The dominant paradigm of generative model use is increasingly agentic: systems plan, invoke tools, observe intermediate results, and iterate over long horizons to complete complex tasks (Yang et al., 2024; Wang et al., 2025). As agentic capabilities increase, so does the range of settings in which a given system might be deployed – and, with it, the number of benchmarks on which the system must be evaluated before it can be trusted with consequential work (Kapoor et al., 2026). Agentic evaluation, however, is expensive as each task execution requires a properly sandboxed environment and a single task execution routinely consumes hundreds of thousands of tokens (Jimenez et al., 2024; Terminal-Bench Team, 2026) and can regularly exceed \$2,000 (Kapoor et al., 2026). Given this cost, efficient evaluation is necessary to increase the evaluation coverage of an agentic system.

Task-efficient evaluation methods (e.g., benchmark compression and anchor-item selection (Vivek et al., 2024; Maia Polo et al., 2024), item response theory and adaptive testing (Kipnis et al., 2025; Truong et al., 2025), leveraging cached responses (Helm et al., 2026a), etc.) address this problem by finding structure in item-level information available from previously-evaluated reference systems to decrease the cost of evaluating a new system. Some task-efficient evaluation methods require only the item-level scores from reference systems (Vivek et al., 2024; Kipnis et al., 2025). Others leverage the content of cached artifacts, such as natural language responses or residual streams, to measure the similarity of systems on a set of tasks (Duderstadt et al., 2023; Helm et al., 2026b;a). Notably, agentic workflows often produce artifacts when executing a task – file diffs, tool calls, intermediate plans, and error messages – recorded together in an execution trace that can be used to this effect. Unfortunately, existing methods for using cached artifacts rely on off-the-shelf embedding functions that are not immediately well-suited for representing traces.

We make two primary contributions:

Table 1: Mean Absolute Error (MAE) of benchmark score estimators for SWE-bench Verified and Terminal-Bench 2.0 under leave-one-family-out evaluation and different task selection protocols (random & adaptive). **Bolded** values are the best method for a given benchmark, task selection, and number of tasks. Underlined values are statistically significant ($\alpha = 0.05$) comparisons to IRT per the two-sided Wilcoxon signed-rank test. The TRUBRIC and IRT blend – which takes advantage of both trace geometry and per-execution score – is the best or near-best in all settings.

Num. tasks m	SWE-bench Verified						Terminal-Bench 2.0					
	random			adaptive			random			adaptive		
	1	5	20	1	5	20	1	5	20	1	5	20
sample score	0.442	0.175	0.082	0.380	0.151	0.072	0.396	0.164	0.073	0.360	0.124	0.132
IRT	0.127	0.088	0.048	0.080	0.062	0.034	0.137	0.097	0.052	0.112	0.067	0.039
raw trace	<u>0.099</u>	0.093	0.089	0.096	0.083	0.081	0.135	0.126	0.120	0.128	0.125	0.113
generic	<u>0.079</u>	<u>0.071</u>	0.068	0.074	0.070	0.069	0.139	0.127	0.115	0.144	0.130	0.117
trubric	<u>0.081</u>	<u>0.069</u>	0.060	0.068	0.056	0.060	<u>0.127</u>	0.107	0.093	0.110	0.092	0.084
IRT + trubric	<u>0.081</u>	<u>0.063</u>	<u>0.043</u>	<u>0.062</u>	<u>0.048</u>	<u>0.034</u>	<u>0.122</u>	<u>0.090</u>	<u>0.050</u>	<u>0.099</u>	<u>0.059</u>	<u>0.037</u>

1. **TRUBRIC trace representations.** We introduce TRUBRIC (*trace rubric*): a trace representation constructed by extracting a task specific structured summary of each trace with an LLM and embedding the summary with a standard text encoder. We show that TRUBRIC embeddings preserve information relevant to the task and behavior of the system while minimizing system-, model family-, and harness- specific signal across seven modern text encoders.

2. **Task-efficient evaluation using cached traces.** Given the TRUBRIC embeddings from a set of tasks for a target system and a collection of reference systems, we evaluate their utility for predicting the target system’s full-benchmark score on SWE-bench Verified and Terminal-Bench 2.0 under three different agentic system evaluation protocols. We find that TRUBRIC-based estimators, combined with an item-response model, reduce single-task mean absolute error from 0.13 to 0.08 ($p < 10^{-4}$) and from 0.14 to 0.12 ($p < 10^{-3}$) on the two benchmarks, respectively, and are nearly always better than baselines at any task budget (Table 1).

1.1 PROBLEM STATEMENT

An agentic system $f \in \mathcal{F}$ is a random mapping from a task space $\mathcal{Q}$ to a trace space $\mathcal{X}$: given a task $q \in \mathcal{Q}$, the system plans, invokes tools, and iterates until it terminates. A benchmark is a fixed collection of tasks $Q^* = \{q_1, \dots, q_M\} \subset \mathcal{Q}$ together with an execution harness and an automatic grader. Executing f on a task q produces a trace $x = f(q) \in \mathcal{X}$: the recorded log of the system’s actions, observations, and outputs – file diffs, tool calls, intermediate plans, error messages – generated while attempting the task. Each execution is evaluated by the benchmark’s trace-level scoring function $s : \mathcal{X} \rightarrow \{0, 1\}$, and the system is evaluated by its benchmark score

$$y(f) = \frac{1}{M} \sum_{j=1}^M \mathbb{E}[s(f(q_j))],$$

where the expectation is with respect to the randomness of f . We write y for $y(f)$ when the system is clear from context.

When evaluating a new system, we typically have access to the execution records of n previously-evaluated reference systems $f_1, \dots, f_n$ – both the traces and the trace-level scores on every task:

$$\mathcal{D} = \left\{ \{(x_{ij}, y_{ij})\}_{j=1}^M \right\}_{i=1}^n,$$

where $x_{ij} \sim f_i(q_j)$, $y_{ij} = s(x_{ij})$, and the benchmark score of reference system f_i is $y_i = \frac{1}{M} \sum_{j=1}^M y_{ij}$. Such reference caches exist publicly at scale.¹ For the target system f , we observe traces (and, optionally, trace-level scores) on only a subset $Q \subset Q^*$ of $|Q| = m \ll M$ tasks. Our goal is an estimate $\hat{y}$ of y from the target’s m executions together with $\mathcal{D}$.

¹The SWE-bench Verified leaderboard archive contains traces and outcomes for over one hundred systems; we estimate it embodies \$50K–\$100K of agent compute contributed by submitters.

2 METHOD

3 TRACE REPRESENTATIONS ENCODE CONFOUNDERS

4 EFFICIENT EVALUATION

5 RELATED WORK

6 DISCUSSION

REFERENCES

- Brandon Duderstadt, Hayden S. Helm, and Carey E. Priebe. Comparing foundation models using data kernels. *arXiv preprint arXiv:2305.05126*, 2023.
- Hayden Helm, Johnson, and Carey Priebe. Query-efficient model evaluation using cached responses. In *International Conference on Machine Learning (ICML)*, 2026a. arXiv:2605.07096.
- Hayden Helm, Xiaodong Liu, and Weiwei Yang. Jailbreak susceptibility prediction and mitigation via the behavioral geometry of models. *arXiv preprint arXiv:2605.26409*, 2026b.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- Sayash Kapoor, Benedikt Stroebel, et al. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.11977.
- Alex Kipnis et al. metabench: A sparse benchmark to measure general ability in large language models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.12844.
- Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14992.
- Terminal-Bench Team. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- Sang Truong et al. Reliable and efficient amortized model-based evaluation. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.13335.
- Rajan Vivek, Kawin Ethayarajh, Diyi Yang, and Douwe Kiela. Anchor points: Benchmarking models with much fewer examples. In *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024. arXiv:2309.08638.
- Xingyao Wang et al. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.16741.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.